

Code Quote — Regarde

Subject-Side Consent — v0 Prototype

Quote date: June 6, 2026
Money: US dollars

Good through: July 6, 2026
If we start: the day the NDA & SOW are signed

From	For
Aesthetic.Computer	Thinkso, Inc.
Authored by Jeffrey Alan Scudder	Jayson Mittman
Billed under Aesthetic Inc.	Regarde — subject-side consent infrastructure

The short version

A working v0 of the subject-side mechanism — the part the white paper makes the load-bearing argument for. A dispatch router that decides on the named person’s terms, a verifiable receipt that anyone can check offline, and a small app that walks a request through the whole flow. It runs on **your** stack with no live tie to anything of mine, and the deliverable is yours outright. Two weeks from kickoff. The price is **\$2,750**, paid in two even halves.

What you're getting

Piece	Cost
Interface sketch — the canonical payload & receipt schema, one page, settled first	\$350.00
Dispatch router — subject-keyed, the four operation classes, allow / deny / refuse	\$550.00
Verifiable receipt — hash-chained, canonicalized, checks offline	\$550.00
Security & infrastructure — receipt crypto hardened, a private isolated build environment, prototype kept to private use	\$550.00
Walkthrough app — drives one request end to end, request → decision → receipt	\$450.00
Proof-of-concept workflows & handoff — the demo flows, code handed over with notes	\$300.00

Two weeks from kickoff • runs on your stack • work-for-hire, assigned to you • optional managed hosting (p. 4)

Total\$2,750.00

How payment works

Part	When	Amount
1 — to begin	NDA & SOW signed, schema work starts	\$1,375.00
2 — on delivery	v0 handed over, running on your stack	\$1,375.00

The whole thing, without the jargon

Today, when a company wants to use someone’s face or voice, it asks whatever middleman happens to be holding it — and the person never gets a say. The idea in your paper flips that around: the request goes to the **person** first, and is answered on their terms. What I’m building is the working version of that flip — small, but real, and something you can actually run.

What you actually get

The piece	In plain words
Interface sketch	The one-page blueprint we both sign off on before anything gets built.
Dispatch router	The gatekeeper. It takes each request, works out what kind it is, and answers on the person’s terms — yes, no, or “I won’t even say.”
Verifiable receipt	A tamper-proof receipt for every answer. Anyone can check later that it’s real — without calling you or me.
Security & infrastructure	The locks on all of it, built somewhere private and kept to you.
Walkthrough app	A little screen that lets you watch one request go through, start to finish.
Proof-of-concept workflows	A couple of worked examples, so it’s something you run, not just read.

What the v0 is

A handful of small parts that fit together into one runnable flow:

✓	Dispatch router — the policy decision point, keyed to the named person. It reads a request, sorts it into one of the four classes (prediction, verification, data-operation, probe), and returns allow, deny, or refuse.
✓	Verifiable receipt — every decision emits a hash-chained, signed receipt. Issuer and verifier produce byte-identical canonical form from the same payload, so a receipt checks out offline with no call back to me.
✓	Security & infrastructure — a hardening pass over the receipt mechanism (key handling, signature scheme, resistance to replay and forgery, closing the refusal side-channel), built on a private, isolated instance and kept to your private use throughout.
✓	Walkthrough app — a small interface that drives a single request through the whole path so the mechanism is something you can poke at, not just read.
✓	Proof-of-concept workflows — the canonical flows scripted as demos, including the refusal path (refusal-as-side-channel is the one to get right).

What runs where

The v0 is built to run **entirely on your stack**. No live dependency on the Aesthetic.Computer PDS or on anything I host — it stands on its own, and you can run it after I walk away. The first thing we settle is the **interface sketch**: the canonical payload and receipt schema on one page. Schema before metaphor — it's the right first artifact and it doubles as our scope contract.

The one thing to pin first

The verify-offline property lives or dies on determinism, so before any code the canonicalization is named and frozen. I'd propose **JCS / RFC 8785** as the default unless your filings point somewhere else — agree that surface up front and the rest of the build is clean.

Terms

✓	Work-for-hire & assignment — the deliverable, in full, is assigned to you. No retained claim, no shared ownership.
✓	NDA precedes schema — signed before we trade any schema; the interface sketch alone gets us into it, so it goes first.
✓	Consulting agreement & SOW to follow — this quote sets price and shape; your counsel's paper carries the legal terms.
✓	Runs on your stack — delivered as code with handoff notes, no live tie to my infrastructure.
✓	Private use — the prototype is built on a private, isolated instance and kept to your private use; never published as a public Aesthetic.Computer piece.
✓	Canonicalization — JCS / RFC 8785 proposed; confirm or redirect before kickoff.

Code Quote — Regarde

Managed Hosting

This part is **optional** and separate from the \$2,750 build on the other pages. The build is yours and runs on your stack — you don't need me to host it. But if you'd rather it just run, I'll stand it up as **Aesthetic.Computer managed secure hosting** on behalf of Thinkso: a private, isolated instance, brought online and kept live, so there's always something to demo from during diligence. A flat **\$500** on top of the build — **\$3,250** all in, with hosting handled.

What's included

✓	A private, isolated instance — brought online and kept live.
✓	A domain registered and pointed at it.
✓	Security patches and updates through the first year.
✓	Backups; a revocable, rotatable demo signing key — you generate and hold the keypair, keys never leave Thinkso's custody.
✓	A status check before each diligence meeting, on request.

\$500 / first year

What's covered vs. a separate quote

Covered by hosting	A separate quote
The instance, domain, patching & backups	Anything new built on top — a fresh code quote
Keeping it live through the first year	Year-two renewal (hosting + domain), billed then
Routine updates and status checks	Running it at production scale, or under an SLA

Optional and entirely separate from the \$2,750 build • cancel anytime • the code is yours either way